

Basic (critique your own Stage-1 design) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `design_self_review.md`: a weak design decision caught on paper costs nothing; the same decision caught three weeks into building costs three weeks. This exercise is that pause, done on purpose, before Stage 2. **If not:** unneeded pieces would get built, and missing ones would be found only when a requirement fails in production.

Write your own version first, in `practice/design_self_review.md`, then compare. Read both depths — they're 2 real levels of the same answer.

Since this document's real requirements are handed to you live, this solution walks through the technique on a realistic stand-in example: a capstone with requirements "handle 50,000 requests/day, p95 latency under 2 seconds, cost under \$200/month, must never expose raw customer data to the LLM." (In this example, the capstone design added a fact-checker agent to Project 4's team, to check each draft's claims against the RAG sources.)

Basic Version

Approach 1 — the plain version

Story: the quickest useful check — for each piece, "what would break if I removed it?" **If not:** anything that sounded like good practice would slip in unquestioned.

Go through your design piece by piece and ask: "what would break if I removed this?"

Piece	What would break if removed?	Verdict
Supervisor agent	Nothing routes work between specialists	Keep
Cache in front of retriever	Latency target (2s) likely missed at 50K/day	Keep
Second, parallel fact-checker	Nothing named — no requirement asks for double-checking	Cut, or write down why you're keeping it

Piece	What would break if removed?	Verdict
Field-level redaction before sending data to the LLM	The "never expose raw customer data" requirement	Keep
Admin dashboard with 12 chart types	Nothing named in requirements	Cut down to what's actually needed for monitoring

This simple pass alone usually finds 1-2 things that snuck in because they seemed like good practice, not because a requirement asked for them.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — the two-direction check

Story: check both directions — every component needs a requirement, and every requirement needs a component. The second direction finds what's *missing*, which is easier to overlook than what's extra. **If not:** the cost limit here would have no component enforcing it, and nothing on the page would look wrong.

Run the full two-direction traceability check.

Direction 1 — every component traces to a requirement:

Component	Requirement it satisfies	Verdict
Supervisor pattern (Doc11) with dynamic routing	Functional: fact-check failures must route back to the writer, a fixed pipeline can't do this	Keep — justified, not just "the standard pattern"
Redis cache in front of the retriever	Non-functional: p95 latency under 2s at 50K req/day; re-embedding repeated queries is the dominant cost	Keep
Second, parallel fact-checker agent	None found	Cut. If a later reliability audit shows single-checker accuracy is insufficient, this becomes justified — right now it's speculative
Field-level redaction layer before any LLM call	Security: "must never expose raw customer data to the LLM"	Keep — this is a hard requirement, not a nice-to-have
Full admin dashboard, 12 chart types	Nothing stated — "watching/monitoring" is listed generically, not with this level of detail	Cut to 3-4 charts that map to the actual non-functional targets (latency, cost, error rate); the rest is speculative polish

Direction 2 — every non-functional requirement has something addressing it (the reverse check):

Requirement	What in the design addresses it?	Gap?
p95 latency under 2s	Cache + async tool calls	Covered
Cost under \$200/month	<i>(nothing explicit yet)</i>	Gap — no component in the current design tracks or caps per-request cost. This needs to be added, not just noted.
Never expose raw customer data to the LLM	Redaction layer	Covered
Reliability target <i>(not yet specified)</i>	<i>(depends on live handoff)</i>	Flag as open until the number is known

Why this matters: the forward check (Direction 1) catches unjustified additions — the failure mode of over-building. The reverse check (Direction 2) catches missing coverage — the failure mode of under-building, which is just as common and easier to miss because nothing "looks wrong" about a design that's simply incomplete. Finding the cost-tracking gap here, in writing, before any code exists, is exactly the value this exercise is meant to deliver.

Approach 2 — a bias pass on your own requirement column

Story: the table is only as honest as the person filling it in — and that person already chose every component. This pass re-checks the requirement column against the actual document, starting with the rows you filled in fastest. **If not:** a requirement written *after* the component would pass as a real one.

The two-direction table catches unjustified and missing pieces — but it assumes the "requirement" column is honest. It might not be, because the same person filling it in is the person who already chose every component. Run a bias pass over the table above:

Component	Row from Direction 1	Bias check: was this requirement real and independent, or reverse-engineered?
Supervisor with dynamic routing	"Fact-check failures must route back to the writer"	Real — this came from the functional requirements in the live handoff, stated before the pattern was chosen. Confident KEEP.
Redis cache	"p95 latency under 2s at 50K req/day"	Real — this is a numeric non-functional requirement stated up front, not inferred after the fact. Confident KEEP.
Field-level redaction layer	"must never expose raw customer data to the LLM"	Suspect at first pass — this is exactly the kind of row that's easy to write confidently <i>after</i> designing the layer, because "customer data is sensitive" sounds self-evidently true. Checked against the actual written security requirements: it IS stated verbatim. Confirmed real, not rationalized — but it

Component	Row from Direction 1	Bias check: was this requirement real and independent, or reverse-engineered?
		only became trustworthy after being checked, not because it felt obviously true.
Admin dashboard, 12 charts	"watching/monitoring"	Caught immediately — the gap between a one-word generic requirement and a specific 12-chart build is too large to be a real match. This is what a rationalization looks like when it's still bad at hiding.

The dangerous row here isn't the dashboard — that one's an easy catch. It's the redaction layer: the verdict ("KEEP") was correct, but if the check had stopped at "can I name a requirement" instead of "was this requirement written down independently of the component," a fabricated-sounding but ultimately real requirement and an actually-fabricated one would have looked identical on the page. The discipline is checking *every* confident row against the source document, not just the ones that feel shaky.

Why this is the harder half of the exercise: Direction 1 and Direction 2 are mechanical — fill in a table, look for gaps. This pass isn't mechanical, because the failure mode is your own reasoning, and your own reasoning doesn't announce itself as biased. The tell is speed and confidence: a row you filled in instantly, without hesitation, for a component you already liked, is exactly the row worth re-checking against the actual requirements document instead of your memory of "why I built it that way."

Which one should you actually run? Do the Basic pass first — fast, catches the obvious cases. Run Approach 1's two-direction check before calling Stage 1 done — it's the harder discipline most people skip under time pressure, and it catches real gaps as often as real excess. Run Approach 2's bias pass specifically on whichever rows you filled in fastest and most confidently — those are statistically the ones worth a second look, precisely because confidence and correctness aren't the same thing when you're grading your own work.